Integrating Project Proposal Document

Proposed project: SIMD-accelerated similarity search for numerical data

Description

Similarity search is a way to find the items in a collection that are most like a given query. Instead of looking for an exact match, it compares the query with each item and ranks them by similarity.

In our project, we'll represent the query and each item as a vector, which is a list of numbers. We'll compare them using cosine similarity. It measures how closely two vectors point in the same direction; a higher score means the vectors are more similar.

We'll calculate a score between the query and every item, then rank the items from highest score to lowest. The highest-ranking items are the closest matches. Since we're using generated vectors, our project will demonstrate how the comparison and ranking work.

Deliverables

There are 3 main deliverables we will implement. These are a normal C baseline, an SIMD parallel version, and synthetic numerical data. To evaluate our work, there are

2 evaluation criteria. These are checking that both versions produce the same ranking and measuring the speedup.

The first deliverable we will implement is the normal C baseline. This program will calculate cosine similarity between a single query vector and multiple candidate vectors sequentially using standard C loops. It can compare one query vector against many vectors and rank the most similar results. This shows how standard scalar code performs without vector hardware, providing our reference point for both ranking and runtime.

On the other hand, the second deliverable we will implement is the SIMD version. Cosine similarity requires multiple dot products and vector magnitude calculations. The reason for the move from the normal C baseline is its inability to process vector dimensions in parallel. The SIMD version will use hardware vector registers to compute the dot products and simultaneously across multiple data lanes. This is our way of reducing execution time while producing the same ranking as normal C version.

The third deliverable we will create is the synthetic numerical data. We will only be using generated data.

The first way of evaluation is by checking output ranking. We will compare the ranked list of documents produced by the normal C baseline against the ranked list produced by the SIMD version. Both implementations must produce the exact same ranking. This will indicate the correctness of the SIMD optimization.

Second is measuring speedup. We will measure the execution time of both versions. We will calculate the speedup through division of C baseline runtime over the

SIMD version. This shows how much runtime the SIMD version saves over the scalar baseline.